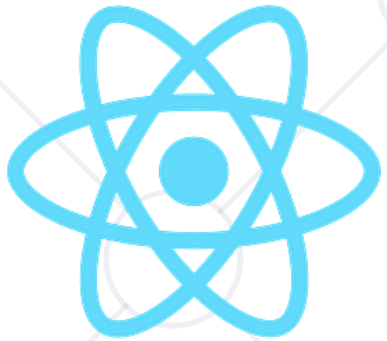
Native Features

The bridge between code and hardware



SoftUni Team

Technical Trainers

Software University

SoftUni

Software University

https://softuni.bg

Table of Contents

Native Features Overview

What Are Native Features?

- Native features are capabilities provided directly by the mobile operating system
  - Exposed by **iOS** and **Android**
  - Provide access to **hardware** and **system services**
  - Not available in standard web apps
  - Often require **user permissions**
  - Usually **asynchronous**

Native features are what make mobile apps truly mobile.

Why Native Features Matter

- Native features define the advantage of mobile apps over the web
  - Camera, sensors, location
  - System **integrations** (share, clipboard)
  - **Notifications** and **background** behavior
  - **Hardware-level** performance
  - Deeper user **trust** and **engagement**

Without native features, mobile apps lose their purpose.

React Native and the Native Layer

- React Native connects JavaScript to **native APIs**
  - **JS** defines behavior
  - Native code **executes** the work
  - APIs are **promise-based**
  - Platform differences are **expected**

React Native orchestrates native code — it doesn't replace it.

Expo vs React Native CLI (Native Access)

- How native features are accessed depends on your setup
- **Expo**
  - Prebuilt native APIs
  - Minimal configuration
  - Faster development

Expo optimizes speed.

- **React Native CLI**
  - Full native control
  - Manual setup
  - Required for custom native code

CLI optimizes control.

Permissions & System Access

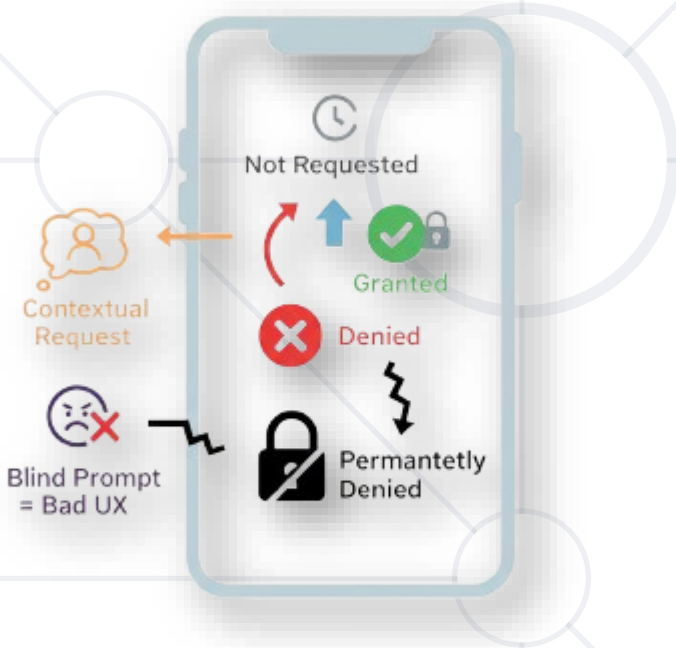
Permissions as a Core Concept

- Permissions gate **access** to native features
  - Camera, location, notifications
  - Controlled by the **OS**
  - Requested at runtime
  - User-controlled

Permissions are not technical — they are trust contracts.

Permission States and User Flow

- Permissions have **multiple** states
  - Not requested
  - Granted
  - Denied
  - Permanently denied
  - Apps must react to each state
  - Requests should be contextual
  - Blind permission prompts hurt UX



Ask only when the user understands why.

Platform Differences in Permissions

Software University

- iOS and Android handle permissions differently.
  - iOS is **stricter and more explicit**
  - Android allows **partial and grouped permissions**
  - Some permissions reset **automatically**
  - Background permissions are treated **differently**

Cross-platform does not mean identical behavior.

Handling Permission Denial

- Permission denial is a valid outcome
  - Never **crash** or block the app
  - **Explain** why the feature matters
  - **Provide** alternative flows
  - **Guide** users to settings if needed

A denied permission is not an error.

Expo Native APIs

Sharing with the System Share Sheet

- **Trigger** the native share dialog
  - **Share** text, links, or files
  - Uses **OS-native** UI
  - Familiar to users
  - No custom **UI** needed

Native sharing feels instantly trustworthy.

Clipboard Access

- Interact with system **clipboard**
  - **Copy** text programmatically
  - Read clipboard **content**
  - Useful for promo code, link, OTP

Small feature, big usability win.

Haptics

- Haptics provide physical feedback through vibration and touch
  - Triggered by **user interaction**
  - Uses the device's vibration motor
  - Different **intensity levels** (light, medium, heavy)
  - Platform-supported native behavior
  - Subtle but powerful **UX enhancer**

Haptics use Cases

- Common **use cases**
  - Button presses
  - Success / error feedback
  - Confirming important actions
  - Reinforcing gestures

Good haptics feel natural. Bad haptics feel distracting.

Using Expo APIs Safely

- System APIs require **care**
  - Handle permissions **explicitly**
  - Expect **failures**
  - **Avoid** silent access
  - **Respect** platform guidelines

Native power requires discipline.

Media & Device Input

Camera Access

- Using the device **camera** directly using **expo-camera**
  - Take photos or record video
  - Front or rear camera
  - Requires explicit permission
  - Common **use cases**
    - Profile images
    - QR scanning
    - Document capture

Camera access is one of the most common native needs.

Image Picker

- Selecting **existing** media from the device using **expo-image-picker**
  - Access **photos** and **videos**
  - **Less intrusive** than camera
  - Still permission-based
  - Often used as an **alternative** option

Let users choose how to provide media.

Document Picker

- **Document Picker** allows users to select files from the device storage using **expo-document-picker**
  - Access files **outside** the media library
  - Supports **documents**, **PDFs**, and **other** file types
  - Uses the system file picker **UI**
  - **No direct** file system browsing
  - Permission handled by the **OS**

Document Picker use Cases

- Common use cases:
  - **Uploading** documents
  - Attaching files to forms
  - **Importing** PDFs or contracts
  - Selecting files for **processing**

The document picker gives access to files without owning the file system.

File System Access

- File System access allows apps to **read and write files** within their own sandbox
  - App-specific storage **only**
  - **No access** to arbitrary user files
  - Used for **caching** and **persistence**
  - Works **offline**
  - Cleared on app **uninstall**

File System Access use Cases

- Common **use cases**:
  - **Storing** downloaded data
  - **Caching** images or media
  - **Saving** user-generated content
  - Temporary **file storage**

Mobile apps don't own the device — they own their sandbox.

Location & Device Context

What Is Location Access?

- Location provides geographic context
  - **GPS-based** positioning
  - Can be **precise** or **approximate**
  - Real-time or one-time
  - Heavy on privacy concerns

Location is powerful — and sensitive.

Foreground vs Background Location

- Location access depends on app state
  - **Foreground**
    - App is visible
    - More permissive
  - **Background**
    - App not visible
    - Heavily restricted
    - Requires extra justification

Background location is a high-trust feature.

Accuracy, Battery, and Trade-offs

- Location precision affects **performance**
  - **High** accuracy → **higher** battery drain
  - **Low** accuracy → **faster** and **cheaper**
  - Choose based on real needs
  - Never **default** to **maximum precision**

Precision is a cost.

Location Permissions and Privacy

Software University

- Location permissions **demand transparency**
  - Explain usage **clearly**
  - **Avoid** vague descriptions
  - Respect **OS rules**
  - Consider **legal implications**

Location misuse breaks user trust permanently.

Geocoding & Reverse Geocoding

- **Geocoding** converts between geographic coordinates and human-readable locations
  - **Geocoding**: address → latitude / longitude
  - **Reverse geocoding**: latitude / longitude → address
  - Common use cases:
    - Displaying **addresses** from **GPS coordinates**
    - Searching locations by **name**
    - Showing user-friendly location labels
    - Enhancing maps and location-based features

Coordinates are for machines — addresses are for humans.

Notifications

What Are Push Notifications?

Software University

- **Notifications** allow apps to communicate externally
  - Delivered by the **OS**
  - Shown outside the **app**
  - Time-sensitive messages
  - Can **re-engage** users

Notifications are interruptions — treat them carefully.

Local vs Remote Notifications

- Two types of notifications

Local

- Scheduled on the device
- No backend required
- Used for reminders

Remote (Push)

- Sent from a server
- Requires push services
- Used for real-time updates

Not all notifications need a backend.

Notification Permissions

- Notifications are **permission-protected**
  - **Explicit** user consent
  - Can be **disabled** anytime
  - **Platform-specific** behavior
  - **iOS** is especially strict

Once denied, trust is hard to regain.

App States and Notification Behavior

- Notification **behavior** depends on app state
  - Foreground
  - Background
  - Killed
  - Different handling per state
  - Requires explicit configuration
  - Testing is essential

Notifications behave differently when the app is alive.

Notifications in Expo

- Expo **simplifies** notification setup with **expo-notifications**
  - **Unified** API
  - **Handles** platform differences
  - **Manages** permissions
  - **Integrates** with Expo services

Expo removes a lot of native friction.

Summary

- **Native features** (camera, GPS, haptics) provide hardware access and performance that standard web apps cannot match, serving as the primary advantage of mobile applications.
- **Access to sensitive data** (location, media) is gated by OS-level per missions, these must be handled contextually and gracefully to maintain user trust and prevent app crashes.
- Key capabilities like **Push Notifications**, **File System sandboxing**, and **Device Sensors** allow apps to interact with the user and the OS even when the app is in the background.

Trainings @ Software University (SoftUni)

- Software University – High-Quality Education, Profession and Job for Software Developers
  - softuni.bg
- Software University Foundation
  - softuni.foundation
- Software University @ Facebook
  - facebook.com/SoftwareUniversity

License

- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – https://about.softuni.bg/
- © Software University – https://softuni.bg